Telecom Customer Churn Analysis

Goal: understand why customers leave, which segments are most at risk, and how much revenue churn costs.

Data: `telecom.db` with two tables

- `customer_churn`
- `zip_population`

Import Libraries

In [1]:
```
import pandas as pd
import numpy as np
import sqlite3
import os
```

Import Database

In [2]:
```
# Connect to the Database
conn = sqlite3.connect(r"data\telecom.db")
# Get the list of all tables in the database
tables = pd.read_sql("""SELECT name FROM sqlite_master
            WHERE TYPE = 'table'""", conn)
# Load each table into its own dataframe
for table in tables["name"]:
    globals()[f"df_{table}"] = pd.read_sql(f"SELECT * FROM {table}", conn)
    print(f"Created Dataframe: df_{table}")
# Close the database connection
conn.close()
```

```
Created Dataframe: df_customer_churn
Created Dataframe: df_zip_population
```

Display All Columns in Output

In [3]:
```
pd.set_option("display.max_columns", None)
```

Preview Raw Data

In [4]:
```
# Preview customer data
display(df_customer_churn.head())
# Preview population data
display(df_zip_population.head())
```

	Customer ID	Gender	Age	Married	Number of Dependents	City	Zip Code	Latitude	Longitude
0	0002-ORFBO	Female	37	Yes	0	Frazier Park	93225	34.827662	-118.999073
1	0003-MKNFE	Male	46	No	0	Glendale	91206	34.162515	-118.203869
2	0004-TLHLJ	Male	50	No	0	Costa Mesa	92627	33.645672	-117.922613
3	0011-IGKFF	Male	78	Yes	0	Martinez	94553	38.014457	-122.115432
4	0013-EXCHZ	Female	75	Yes	0	Camarillo	93010	34.227846	-119.079903

	Zip Code	Population
0	90001	54492
1	90002	44586
2	90003	58198
3	90004	67852
4	90005	43019

Data Cleaning

Standardize Column Names

In [5]:
```
# Standardize column names in customer data
df_customer_churn.columns = df_customer_churn.columns.str.strip().str.lower().str.r
# Standardize column names in population data
df_zip_population.columns = df_zip_population.columns.str.strip().str.lower().str.r
# Print cleaned column names
print(df_customer_churn.columns)
print(df_zip_population.columns)
```

```
Index(['customer_id', 'gender', 'age', 'married', 'number_of_dependents',
       'city', 'zip_code', 'latitude', 'longitude', 'number_of_referrals',
       'tenure_in_months', 'offer', 'phone_service',
       'avg_monthly_long_distance_charges', 'multiple_lines',
       'internet_service', 'internet_type', 'avg_monthly_gb_download',
       'online_security', 'online_backup', 'device_protection_plan',
       'premium_tech_support', 'streaming_tv', 'streaming_movies',
       'streaming_music', 'unlimited_data', 'contract', 'paperless_billing',
       'payment_method', 'monthly_charge', 'total_charges', 'total_refunds',
       'total_extra_data_charges', 'total_long_distance_charges',
       'total_revenue', 'customer_status', 'churn_category', 'churn_reason'],
      dtype='str')
Index(['zip_code', 'population'], dtype='str')
```

Review Dataset Info

In [6]:
```
print(df_customer_churn.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 12143 entries, 0 to 12142
Data columns (total 38 columns):
 #   Column                             Non-Null Count  Dtype  
---  ------                             --------------  -----  
 0   customer_id                        12143 non-null  str    
 1   gender                             12143 non-null  str    
 2   age                                12143 non-null  int64  
 3   married                            12143 non-null  str    
 4   number_of_dependents               12143 non-null  int64  
 5   city                               12143 non-null  str    
 6   zip_code                           12143 non-null  int64  
 7   latitude                           12143 non-null  float64
 8   longitude                          12143 non-null  float64
 9   number_of_referrals                12143 non-null  int64  
 10  tenure_in_months                   12143 non-null  int64  
 11  offer                              12143 non-null  str    
 12  phone_service                      12143 non-null  str    
 13  avg_monthly_long_distance_charges  10992 non-null  float64
 14  multiple_lines                     10992 non-null  str    
 15  internet_service                   12143 non-null  str    
 16  internet_type                      9538 non-null   str    
 17  avg_monthly_gb_download            9538 non-null   float64
 18  online_security                    9538 non-null   str    
 19  online_backup                      9538 non-null   str    
 20  device_protection_plan             9538 non-null   str    
 21  premium_tech_support               9538 non-null   str    
 22  streaming_tv                       9538 non-null   str    
 23  streaming_movies                   9538 non-null   str    
 24  streaming_music                    9538 non-null   str    
 25  unlimited_data                     9538 non-null   str    
 26  contract                           12143 non-null  str    
 27  paperless_billing                  12143 non-null  str    
 28  payment_method                     12143 non-null  str    
 29  monthly_charge                     12143 non-null  float64
 30  total_charges                      12143 non-null  float64
 31  total_refunds                      12143 non-null  float64
 32  total_extra_data_charges           12143 non-null  int64  
 33  total_long_distance_charges        12143 non-null  float64
 34  total_revenue                      12143 non-null  float64
 35  customer_status                    12143 non-null  str    
 36  churn_category                     3234 non-null   str    
 37  churn_reason                       3234 non-null   str    
dtypes: float64(9), int64(6), str(23)
memory usage: 3.5 MB
None
```

Check for Duplicates

In [7]:
```
print(f"duplicate customer_id : {df_customer_churn["customer_id"].duplicated().sum(
print(f"duplicate rows : {df_customer_churn.duplicated().sum()}")
print(f"duplicate zip_code : {df_zip_population["zip_code"].duplicated().sum()}")
```

```
duplicate customer_id : 0
duplicate rows : 0
duplicate zip_code : 0
```

Drop Unnecessary Columns

In [8]:
```
df_customer_churn.drop(["gender", "age", "married","number_of_dependents", "latitud
```

Standardize Text Values

In [9]:
```
# Select text columns
str_cols = df_customer_churn.select_dtypes(include = "str").columns.tolist()
# Remove extra spaces from text columns
df_customer_churn[str_cols] = df_customer_churn[str_cols].apply(lambda col : col.st
```

Review Unique Values

In [10]:
```
# Columns to exclude
exclude = ["customer_id", "city", "churn_reason"]
# Select categorical columns
cat_cols = [col for col in str_cols if col not in exclude]
# Show value counts for each categorical column
for col in cat_cols:
    print(f"\n---{col}---")
    print(df_customer_churn[col].value_counts(dropna = False))
```

```
---offer---
offer
None       6716
Offer B    1423
Offer E    1375
Offer D    1040
Offer A     875
Offer C     714
Name: count, dtype: int64

---phone_service---
phone_service
Yes    10992
No      1151
Name: count, dtype: int64

---multiple_lines---
multiple_lines
No     5839
Yes    5153
NaN    1151
Name: count, dtype: int64

---internet_service---
internet_service
Yes    9538
No     2605
Name: count, dtype: int64

---internet_type---
internet_type
Fiber Optic    5257
DSL            2874
NaN            2605
Cable          1407
Name: count, dtype: int64

---online_security---
online_security
No     6077
Yes    3461
NaN    2605
Name: count, dtype: int64

---online_backup---
online_backup
No     5339
Yes    4199
NaN    2605
Name: count, dtype: int64

---device_protection_plan---
device_protection_plan
No     5356
Yes    4182
NaN    2605
```

```
Name: count, dtype: int64

---premium_tech_support---
premium_tech_support
No     6033
Yes    3505
NaN    2605
Name: count, dtype: int64

---streaming_tv---
streaming_tv
No     4818
Yes    4720
NaN    2605
Name: count, dtype: int64

---streaming_movies---
streaming_movies
No     4784
Yes    4754
NaN    2605
Name: count, dtype: int64

---streaming_music---
streaming_music
No     5165
Yes    4373
NaN    2605
Name: count, dtype: int64

---unlimited_data---
unlimited_data
Yes    8215
NaN    2605
No     1323
Name: count, dtype: int64

---contract---
contract
Month-to-Month    6218
Two Year          3219
One Year          2706
Name: count, dtype: int64

---customer_status---
customer_status
Stayed     8152
Churned    3234
Joined      757
Name: count, dtype: int64

---churn_category---
churn_category
NaN               8909
Competitor        1414
Attitude           562
```

```
Dissatisfaction     557
Price               383
Other               318
Name: count, dtype: int64
```

Identify Missing Data

In [11]:
```
print(df_customer_churn.isna().sum())
```

```
customer_id                          0
city                                 0
zip_code                             0
tenure_in_months                     0
offer                                0
phone_service                        0
avg_monthly_long_distance_charges    1151
multiple_lines                       1151
internet_service                     0
internet_type                        2605
avg_monthly_gb_download              2605
online_security                      2605
online_backup                        2605
device_protection_plan               2605
premium_tech_support                 2605
streaming_tv                         2605
streaming_movies                     2605
streaming_music                      2605
unlimited_data                       2605
contract                             0
monthly_charge                       0
total_charges                        0
total_refunds                        0
total_extra_data_charges             0
total_long_distance_charges          0
total_revenue                        0
customer_status                      0
churn_category                       8909
churn_reason                         8909
dtype: int64
```

Check Missing Values in Phone Service vs Related Service Columns

In [12]:
```
phone_cols = ["avg_monthly_long_distance_charges", "multiple_lines"]
for col in phone_cols:
    display("-" * 25 + f"phone_service vs {col}" + "-" * 25)
    display(pd.crosstab(df_customer_churn["phone_service"], df_customer_churn[col].
```

```
'-------------------------phone_service vs avg_monthly_long_distance_charges--------
-----------------'
```

avg_monthly_long_distance_charges	Missing	Present
phone_service		
No	1151	0
Yes	0	10992

'------------------------phone_service vs multiple_lines------------------------'

multiple_lines	Missing	Present
phone_service		
No	1151	0
Yes	0	10992

Check Missing Values in Internet Service vs Related Service Columns

In [13]:
```
internet_cols = df_customer_churn.loc[:, "internet_type" : "unlimited_data"].column
for col in internet_cols:
    display("-" * 25 + f"internet_service vs {col}" + "-" * 25)
    display(pd.crosstab(df_customer_churn["internet_service"], df_customer_churn[co
```

'------------------------internet_service vs internet_type------------------------
'

internet_type	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

'------------------------internet_service vs avg_monthly_gb_download---------------
----------'

avg_monthly_gb_download	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

'------------------------internet_service vs online_security----------------------
--'

online_security	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

'------------------------internet_service vs online_backup------------------------
'

online_backup	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

'------------------------internet_service vs device_protection_plan----------------
---------'

device_protection_plan	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

'------------------------internet_service vs premium_tech_support-----------------
-------'

premium_tech_support	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

'------------------------internet_service vs streaming_tv------------------------'

streaming_tv	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

'------------------------internet_service vs streaming_movies---------------------
---'

streaming_movies	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

'------------------------internet_service vs streaming_music----------------------
--'

streaming_music	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

```
'------------------------internet_service vs unlimited_data-----------------------
-'
```

unlimited_data	Missing	Present
internet_service		
No	2605	0
Yes	0	9538

Check Missing Values in Churn Category vs. Customer Status

In [14]:
```
display(pd.crosstab(df_customer_churn["customer_status"], df_customer_churn["churn_
```

churn_category	Missing	Present
customer_status		
Churned	0	3234
Joined	757	0
Stayed	8152	0

Handle Structural Missing Values

Missing values in the phone and internet columns are **structural, not data errors**. The crosstabs above show that a value is missing *only* when the customer has no phone service (`multiple_lines` , `avg_monthly_long_distance_charges`) or no internet service (internet type, add-ons, `avg_monthly_gb_download`), and never otherwise. The same holds for `churn_category` and `churn_reason` , which are blank for customers who did not churn.

Treatment:

- **Categorical service columns** are filled with "No Phone Service" / "No Internet Service" so they show up as clear categories instead of "(Blank)". The fill is restricted to rows where the parent service is "No", so nothing else can be overwritten.
- **Numeric usage columns** (`avg_monthly_long_distance_charges` , `avg_monthly_gb_download`) stay as NaN. Filling them with 0 would drag down averages for customers who do have the service.

- **Churn category / reason** stay NaN here so churn calculations are unaffected.

In [15]:
```
# Phone-related categorical column
df_customer_churn.loc[df_customer_churn["phone_service"] == "No", "multiple_lines"]

# Internet-related categorical columns
internet_cat_cols = df_customer_churn.loc[:, "internet_type" : "unlimited_data"].co
df_customer_churn.loc[df_customer_churn["internet_service"] == "No", internet_cat_c
```

In [16]:
```
df_customer_churn.isna().sum()[df_customer_churn.isna().sum() > 0]
```

Out[16]:
```
avg_monthly_long_distance_charges    1151
avg_monthly_gb_download              2605
churn_category                       8909
churn_reason                         8909
dtype: int64
```

Check Numeric Distributions and Outliers

In [17]:
```
display(df_customer_churn.describe().T)
```

	count	mean	std	min	25%
zip_code	12143.0	93470.259244	1857.333497	90001.00	92096.000
tenure_in_months	12143.0	32.504323	24.493885	1.00	9.000
avg_monthly_long_distance_charges	10992.0	25.413852	14.268016	1.01	13.020
avg_monthly_gb_download	9538.0	26.290522	19.578359	2.00	13.000
monthly_charge	12143.0	63.868194	31.245335	-10.00	31.645
total_charges	12143.0	2292.709210	2268.706722	16.71	409.125
total_refunds	12143.0	2.054850	8.084369	0.00	0.000
total_extra_data_charges	12143.0	6.803096	25.052237	0.00	0.000
total_long_distance_charges	12143.0	756.635383	853.593445	0.00	73.935
total_revenue	12143.0	3054.092840	2873.812621	18.59	619.245

Inspect Negative Monthly Charges

In [18]:
```
display(df_customer_churn[df_customer_churn["monthly_charge"] < 0].head())
print(f"Rows with negative monthly charge : {(df_customer_churn["monthly_charge"] <
```

	customer_id	city	zip_code	tenure_in_months	offer	phone_service	avg_monthly_l
1	0003-MKNFE	Glendale	91206	9	None	Yes	
32	0052-DCKON	Los Angeles	90063	66	None	Yes	
170	0274-JKUJR	Los Angeles	90040	7	Offer E	No	
232	0361-HJRDX	Sherman Oaks	91403	68	Offer A	No	
336	0495-ZBNGW	Loma Mar	94021	5	None	Yes	

```
Rows with negative monthly charge : 204
```

Check if Negative Charges Relate to Refunds

In [19]:
```
display(pd.crosstab(df_customer_churn["monthly_charge"] < 0, df_customer_churn["tot
```

total_refunds	False	True
monthly_charge		
False	11026	913
True	179	25

Compare Monthly Charge with Estimated Monthly Charge

In [20]:
```
# Keep rows with valid monthly charges
df_valid = df_customer_churn[df_customer_churn["monthly_charge"] > 0].copy()
# Estimate monthly charge from total charges and tenure
df_valid["estimated_monthly_charge"] = df_valid["total_charges"] / df_valid["tenure
# Compare summary statistics of both columns
display(df_valid[["monthly_charge", "estimated_monthly_charge",]].describe())
# Correlation between monthly charge & estimated monthly charge
corr = df_valid["monthly_charge"].corr(df_valid["estimated_monthly_charge"]).round(
print(f"Correlation : {corr}")
```

	monthly_charge	estimated_monthly_charge
count	11939.000000	11939.000000
mean	65.049877	65.047111
std	30.160883	30.239532
min	17.360000	13.775000
25%	36.250000	36.998846
50%	70.750000	70.810000
75%	90.050000	90.243955
max	122.730000	125.322222

```
Correlation : 0.996
```

Fix Negative Monthly Charges

A negative `monthly_charge` is not a valid price. The refund check above shows these rows are **not** explained by refunds, so they look like data-entry errors. Among valid rows, `total_charges / tenure_in_months` tracks `monthly_charge` very closely (correlation shown above), so the invalid values are **imputed** with this estimate instead of dropping the rows or leaving them blank.

In [21]:
```
# Calculate estimated monthly charge
df_customer_churn["est_monthly_charge"] = df_customer_churn["total_charges"] / df_c
# Replace negative values in monthly charge with estimated monthly charge
df_customer_churn.loc[df_customer_churn["monthly_charge"] < 0, "monthly_charge"] =
```

Preview Cleaned Data

In [22]:
```
display(df_customer_churn.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 12143 entries, 0 to 12142
Data columns (total 30 columns):
 #   Column                             Non-Null Count  Dtype
---  ------                             --------------  -----
 0   customer_id                        12143 non-null  str
 1   city                               12143 non-null  str
 2   zip_code                           12143 non-null  int64
 3   tenure_in_months                   12143 non-null  int64
 4   offer                              12143 non-null  str
 5   phone_service                      12143 non-null  str
 6   avg_monthly_long_distance_charges  10992 non-null  float64
 7   multiple_lines                     12143 non-null  str
 8   internet_service                   12143 non-null  str
 9   internet_type                      12143 non-null  str
 10  avg_monthly_gb_download            9538 non-null   float64
 11  online_security                    12143 non-null  str
 12  online_backup                      12143 non-null  str
 13  device_protection_plan             12143 non-null  str
 14  premium_tech_support               12143 non-null  str
 15  streaming_tv                       12143 non-null  str
 16  streaming_movies                   12143 non-null  str
 17  streaming_music                    12143 non-null  str
 18  unlimited_data                     12143 non-null  str
 19  contract                           12143 non-null  str
 20  monthly_charge                     12143 non-null  float64
 21  total_charges                      12143 non-null  float64
 22  total_refunds                      12143 non-null  float64
 23  total_extra_data_charges           12143 non-null  int64
 24  total_long_distance_charges        12143 non-null  float64
 25  total_revenue                      12143 non-null  float64
 26  customer_status                    12143 non-null  str
 27  churn_category                     3234 non-null   str
 28  churn_reason                       3234 non-null   str
 29  est_monthly_charge                 12143 non-null  float64
dtypes: float64(8), int64(3), str(19)
memory usage: 2.8 MB
None
```

Feature Engineering

Create Churn Flag

`Churned` = 1, `Stayed` = 0, `Joined` = NaN (excluded from churn calculations).

In [23]:
```
df_customer_churn["churn_flag"] = df_customer_churn["customer_status"].map({"Churne
```

Create Tenure Group

In [24]:
```
# Define tenure ranges (months)
bins = [0, 12, 24, 36, 48, 60, 72]
# Define a label for each range
labels = ["1-12 months", "13-24 months", "25-36 months", "37-48 months", "49-60 mon
```

```
# Assign each customer to a tenure group
df_customer_churn["tenure_group"] = pd.cut(df_customer_churn["tenure_in_months"], b
```

Merge Tables

In [25]:
```
df = df_customer_churn.merge(df_zip_population, "left", "zip_code", validate = "man
```

Preview Data

In [26]:
```
display(df.head())
```

	customer_id	city	zip_code	tenure_in_months	offer	phone_service	avg_monthly_lo
0	0002-ORFBO	Frazier Park	93225	9	None	Yes	
1	0003-MKNFE	Glendale	91206	9	None	Yes	
2	0004-TLHLJ	Costa Mesa	92627	4	Offer E	Yes	
3	0011-IGKFF	Martinez	94553	13	Offer D	Yes	
4	0013-EXCHZ	Camarillo	93010	3	None	Yes	

Data Analysis

Churn Rate

Churn rate excludes the Joined customers (new this period), so it is churned / (churned + stayed).

In [27]:
```
churn_rate = (df["churn_flag"].mean() * 100).round(1)
print(f"Churn rate : {churn_rate}%")
```

```
Churn rate : 28.4%
```

Retention Rate

In [28]:
```
retention_rate = 100 - churn_rate
print(f"Retention rate : {retention_rate.round(1)}%")
```

```
Retention rate : 71.6%
```

Churn by Tenure

In [29]:
```
# Keep existing customers only (excludes newly joined)
existing_customers = df[df["churn_flag"].notna()]
# Count churned and stayed customers by tenure group
churn_by_tenure = existing_customers.groupby("tenure_group")["customer_status"].val
# Calculate churn rate
churn_by_tenure["churn_rate"] = (churn_by_tenure["Churned"] / (churn_by_tenure["Chu

display(churn_by_tenure)
```

customer_status	Churned	Stayed	churn_rate
tenure_group			
1-12 months	1808	1177	60.6
13-24 months	514	1263	28.9
25-36 months	308	1114	21.7
37-48 months	248	1058	19.0
49-60 months	198	1278	13.4
61-72 months	158	2262	6.5

Churn by Contract

In [30]:
```
# Count customers by contract and customer status
churn_by_contract = pd.crosstab(existing_customers["contract"], existing_customers[
# Calculate churn rate
churn_by_contract["churn_rate"] = (churn_by_contract["Churned"] / (churn_by_contrac

display(churn_by_contract)
```

customer_status	Churned	Stayed	churn_rate
contract			
Month-to-Month	2870	2665	51.9
One Year	277	2393	10.4
Two Year	87	3094	2.7

Churn by Internet Type

In [31]:
```
# Count customers by internet type and customer status
churn_by_internet = pd.crosstab(existing_customers["internet_type"], existing_custo
# Calculate churn rate
```

```
churn_by_internet["churn_rate"] = (churn_by_internet["Churned"] / (churn_by_interne

display(churn_by_internet)
```

customer_status	Churned	Stayed	churn_rate
internet_type			
Cable	362	961	27.4
DSL	549	2124	20.5
Fiber Optic	2117	2968	41.6
No Internet Service	206	2099	8.9

Churn by Offer Type

In [32]:
```
# Count customers by contract and customer status
churn_by_offer = existing_customers.groupby("offer")["customer_status"].value_count
# Calculate churn rate
churn_by_offer["churn_rate"] = (churn_by_offer["Churned"] / (churn_by_offer["Churne

display(churn_by_offer)
```

customer_status	Churned	Stayed	churn_rate
offer			
None	1820	4435	29.1
Offer A	62	813	7.1
Offer B	169	1254	11.9
Offer C	153	561	21.4
Offer D	289	751	27.8
Offer E	741	338	68.7

Churn by Churn Category

In [33]:
```
# Count churned customers by churn category
churn_by_category = existing_customers.groupby(["churn_category", "customer_status"
# Calculate each category's share of total churn
churn_by_category["pct_of_churn"] = (churn_by_category["Churned"] / churn_by_catego

display(churn_by_category)
```

customer_status churn_category	Churned	pct_of_churn
Attitude	562	17.4
Competitor	1414	43.7
Dissatisfaction	557	17.2
Other	318	9.8
Price	383	11.8

Churn by Phone Add-On Services

In [34]:
```
# Customers with phone service
phone_customers = existing_customers[existing_customers["phone_service"] == "Yes"]
# Count customers by multiple lines and customer status
churn_by_phone_service = pd.crosstab(phone_customers["multiple_lines"], phone_custo
# Calculate churn rate
churn_by_phone_service["churn_rate"] = (churn_by_phone_service["Churned"] / (churn_

display(churn_by_phone_service)
```

customer_status multiple_lines	Churned	Stayed	churn_rate
No	1488	3735	28.5
Yes	1457	3616	28.7

Churn by Internet Add-On Services

In [35]:
```
# Customers with internet service
internet_customers = existing_customers[existing_customers["internet_service"] == "

# List of internet add-on services
add_on_services = ["online_security", "online_backup", "device_protection_plan", "p

# Empty list to store the results
churn_tables = []
for service in add_on_services:
    # Count customers by service status and customer status
    churn_table = pd.crosstab(internet_customers[service], internet_customers["cust
    # Calculate churn rate
    churn_table["churn_rate"] = (churn_table["Churned"] / (churn_table["Churned"] +
    # Add service identifier column
    churn_table["service"] = service
    # Rename service column and set service and service status as index
    churn_table = churn_table.reset_index().rename({service : "service_status"}, ax
    # Store each service table
```

```
    churn_tables.append(churn_table)

# Combine all service tables into one dataframe
churn_by_internet_service = pd.concat(churn_tables)

display(churn_by_internet_service)
```

	customer_status	Churned	Stayed	churn_rate
service	service_status			
online_security	No	2516	3178	44.2
	Yes	512	2875	15.1
online_backup	No	2117	2855	42.6
	Yes	911	3198	22.2
device_protection_plan	No	2088	2864	42.2
	Yes	940	3189	22.8
premium_tech_support	No	2500	3144	44.3
	Yes	528	2909	15.4
streaming_tv	No	1620	2820	36.5
	Yes	1408	3233	30.3
streaming_movies	No	1615	2795	36.6
	Yes	1413	3258	30.3
streaming_music	No	1748	3051	36.4
	Yes	1280	3002	29.9
unlimited_data	No	424	819	34.1
	Yes	2604	5234	33.2

Average Customer Tenure

In [36]:
```
average_tenure = df["tenure_in_months"].mean().round(1)

print(f"Average Tenure : {average_tenure} months")
```

```
Average Tenure : 32.5 months
```

Average Revenue per User (ARPU)

In [37]:
```
arpu = (df["total_revenue"].sum() / df["tenure_in_months"].sum()).round(2)

print(f"ARPU : ${arpu}")
```

```
ARPU : $93.96
```

Monthly Revenue Lost from Churned Users

In [38]:
```
monthly_revenue = df["total_revenue"] / df["tenure_in_months"]
lost_revenue = monthly_revenue[df["churn_flag"] == 1].sum().round(2)
existing_revenue = monthly_revenue[df["churn_flag"].notna()].sum().round(2)
print(f"monthly lost revenue : ${lost_revenue:,.2f}")
print(f"monthly revenue of all existing users : ${existing_revenue:,.2f}")
print(f"share of monthly revenue lost to churn : {lost_revenue / existing_revenue *
```

```
monthly lost revenue : $316,404.61
monthly revenue of all existing users : $1,021,727.04
share of monthly revenue lost to churn : 31.0%
```

Export Cleaned Data for Power BI

In [39]:
```
df_export = df.copy()
df_export[["churn_category", "churn_reason"]] = df_export[["churn_category", "churn
df_export.to_csv("telecom_churn_clean.csv", index = False)
print(f"exported {df_export.shape[0]} rows x {df_export.shape[1]} columns")
print(f"\ncolumns that still contain missing values: \n {df_export.isna().sum()[df_
```

```
exported 12143 rows x 33 columns

columns that still contain missing values:
 avg_monthly_long_distance_charges    1151
avg_monthly_gb_download              2605
churn_flag                            757
dtype: int64
```

Key Findings So Far

- **Overall churn is 28.4%** of existing customers (retention 71.6%); `Joined` customers are excluded.
- **Tenure:** The first year is the highest-risk window. Customers with 1-12 months of tenure churn at 60.6%, and the rate falls steadily to 6.5% after five years (61-72 months).
- **Contract:** Month-to-month customers churn at 51.9%, versus 10.4% on one-year and 2.7% on two-year contracts. Month-to-month customers also make up most of all churn (2,870 of 3,234 churned customers).
- **Internet type:** Fiber Optic customers churn most (41.6%), versus Cable (27.4%) and DSL (20.5%).
- **Offers:** Offer E has the highest churn rate (68.7%) and Offer A the lowest (7.1%). Customers with no offer churn at 29.1%, close to the overall average.
- **Churn reasons:** Competitors account for 43.7% of all churn. Attitude (17.4%) and Dissatisfaction (17.2%) are next, and Price is only 11.8%.
- **Add-on services:** Customers with Online Security or Premium Tech Support churn at about 15%, versus about 44% for those without. Online Backup and Device Protection

show a smaller gap (about 22% vs 42%). Unlimited Data makes no real difference (33.2% vs 34.1%). Multiple Lines doesn't matter either (28.7% vs 28.5%).